

Incremental and Semi-Supervised Learning

Ambient Intelligence and Domotics

Master Degree: Artificial Intelligence for Science and Technology

Gabriele Civitarese

EveryWare Lab, Department of Computer Science

Università degli Studi di Milano

Copyright

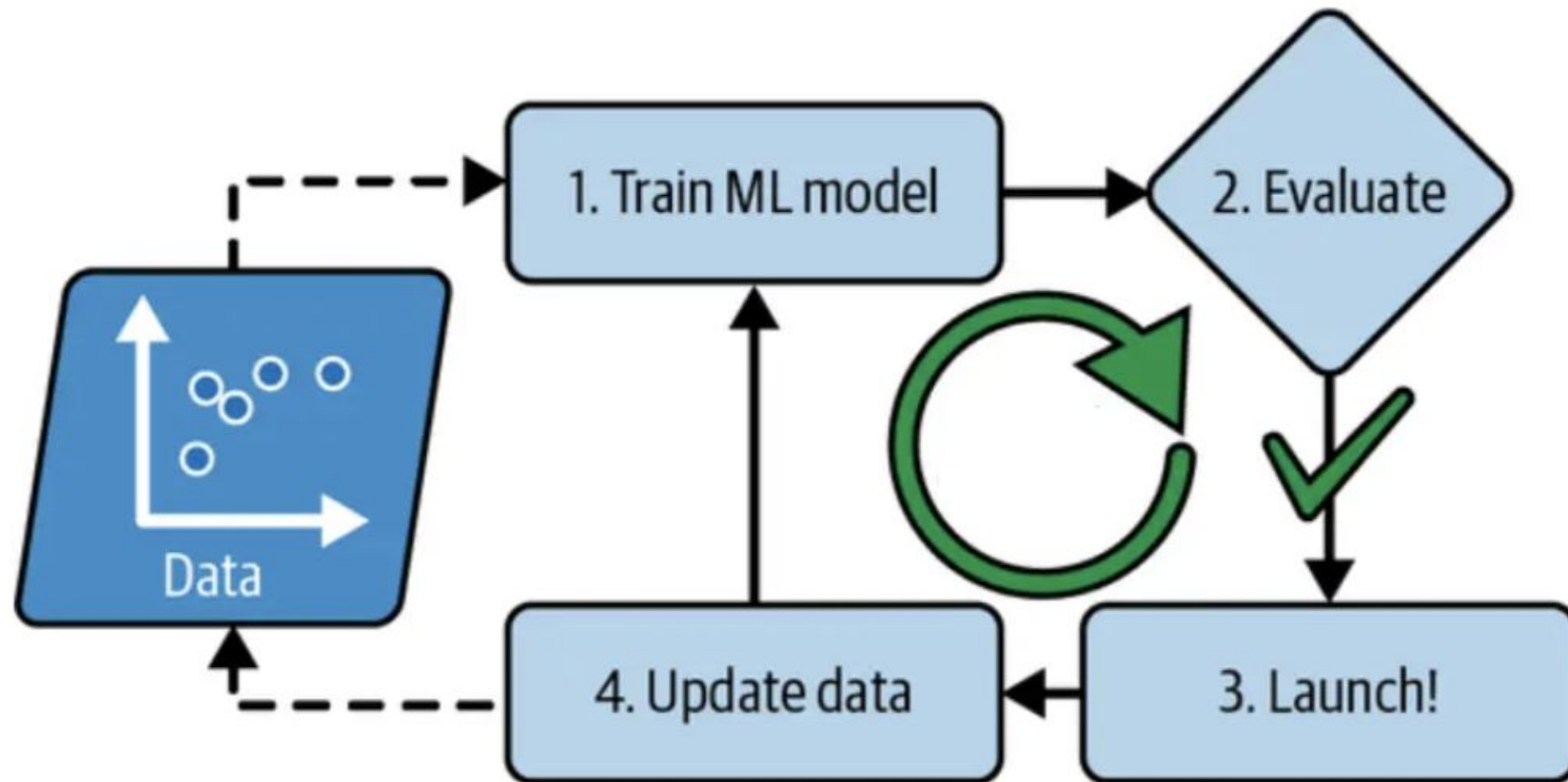
These slides are from the teachers of this course. All the material is subject to copyright and cannot be redistributed without consent of the copyright holder. The same holds for audio and video-recordings of the classes of this course.

Disclaimer

- In this lecture, we will use as driving example Human Activity Recognition, since it provides a concrete scenario that has been extensively studied in the literature
- However, these concepts may be applied to many tasks related to the application of AI methods to sensor data

Static Models vs Adaptive Models

Static Models



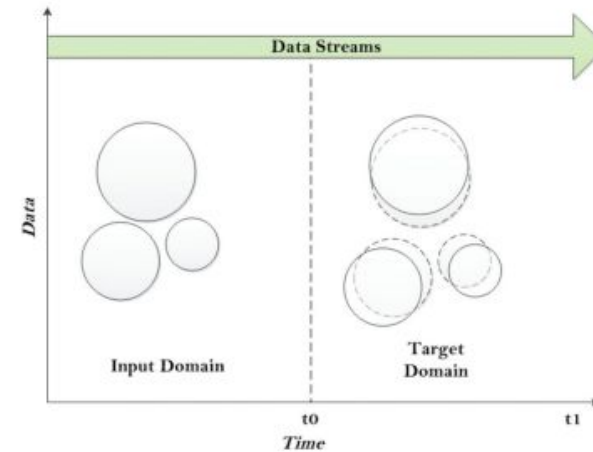
<https://vitalflux.com/difference-between-online-batch-learning/>

Problems of a “static” model

- So far, we always considered “static” models:
 - they are trained once
 - they consider a predetermined set of classes
 - they can not evolve considering new classes or new sensor patterns
- However, after the model is trained, temporal data streams may continuously evolve
- Possible issues:
 - the pre-trained model requires personalization on the specific subject
 - new classes may arise (e.g., new activities)

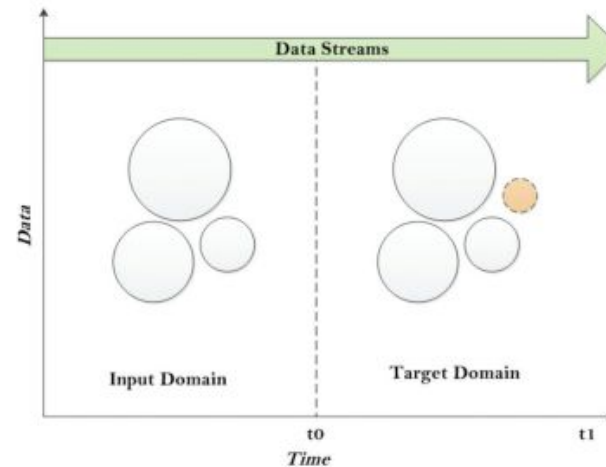
Changes in Data Streams

- **Concept Drift:** the target subject may perform activities with some differences (e.g., young vs elderly)



(a) Concept Drift

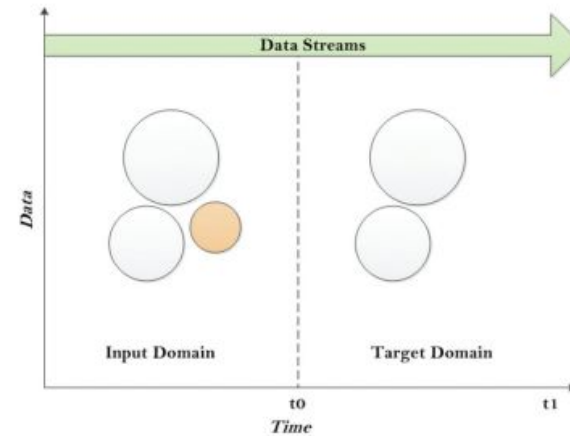
- **Concept Evolution:** new patterns emerge from data: new activities or new ways of performing an activity.



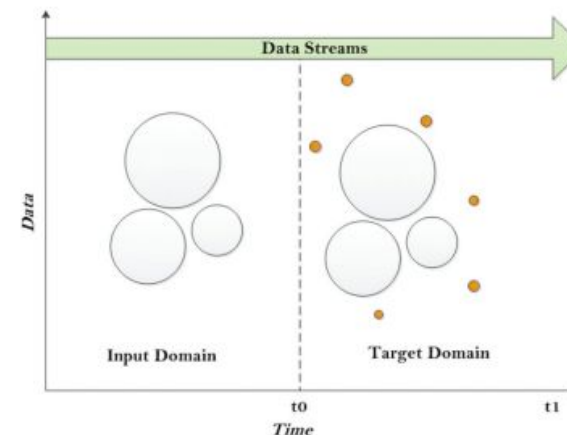
(b) Concept Evolution

Changes in Data Streams (cont.)

- **Concept forgetting:** to improve both accuracy and efficiency, it is crucial to forget patterns if not representative of the target user.
- **Outliers/anomalies:** new data may present sporadic samples particularly distant from learned patterns. It is crucial to distinguish outliers from anomalies.



(c) Concept Forgetting



(d) Outliers/Anomalies

Why not re-training from scratch?

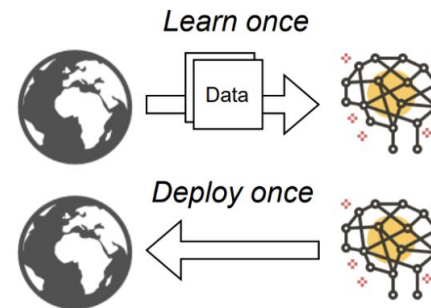
- In principle, a possibility is to periodically re-train the model as new labelled data arrives from the stream
- This solution is not scalable in large scale real-world scenarios, since it requires:
 - to continuously store labeled data (original training set + new data)
 - to periodically re-train the model, which may be computationally costly and can not adapt quickly to changes in the stream



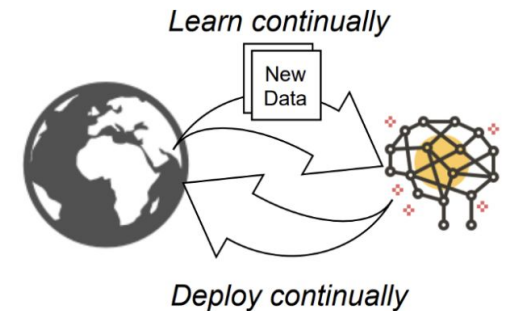
Incremental learning

- Periodically updating the classifier with new mini-batches of training data
 - No need for re-training!
- The model can continuously learn from new data and improve itself:
 - new patterns
 - new classes
 - new domains
 - etcetera

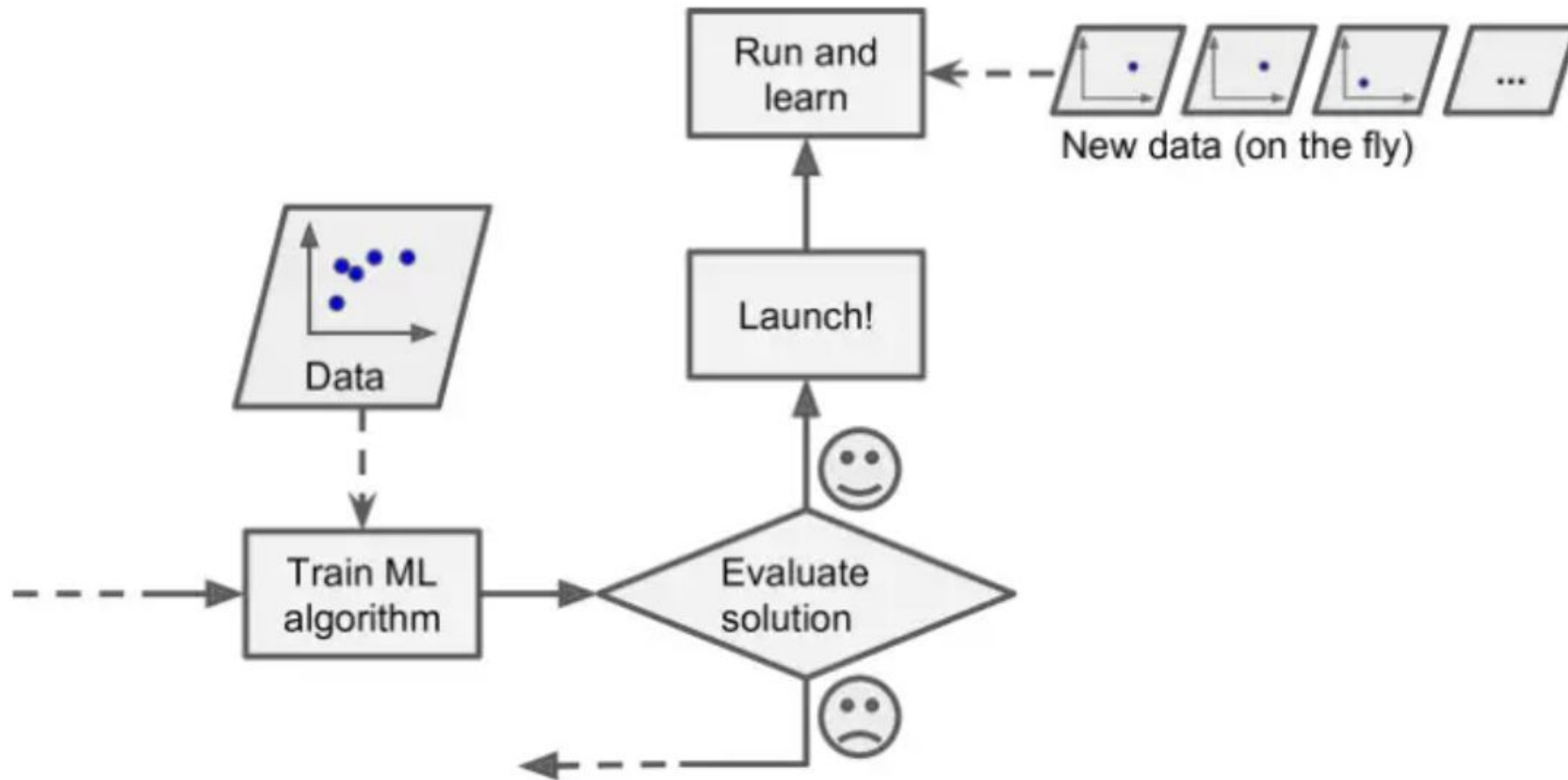
Static ML



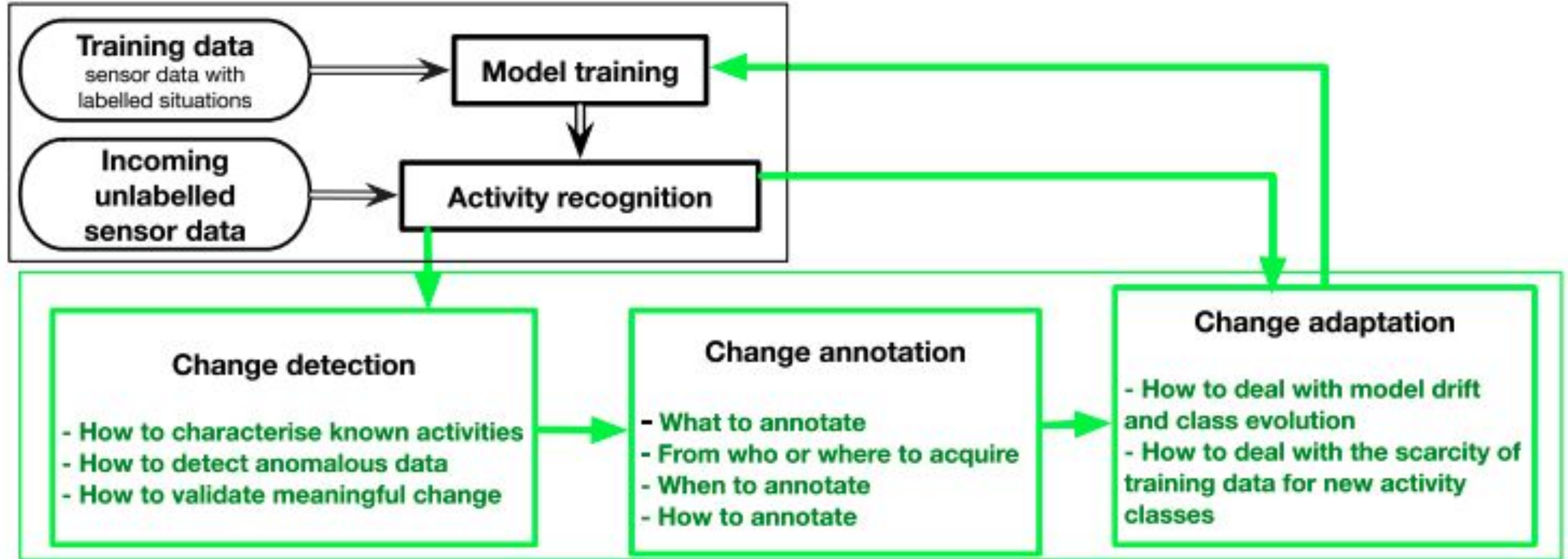
Adaptive ML



Incremental (or online) Learning



“Lifelong” learning: the challenges



[Ye, J., Dobson, S., & Zambonelli, F. (2019). Lifelong learning in sensor-based human activity recognition. IEEE Pervasive Computing, 18(3), 49-58.]

Continual Learning

Continuously Learning from Data

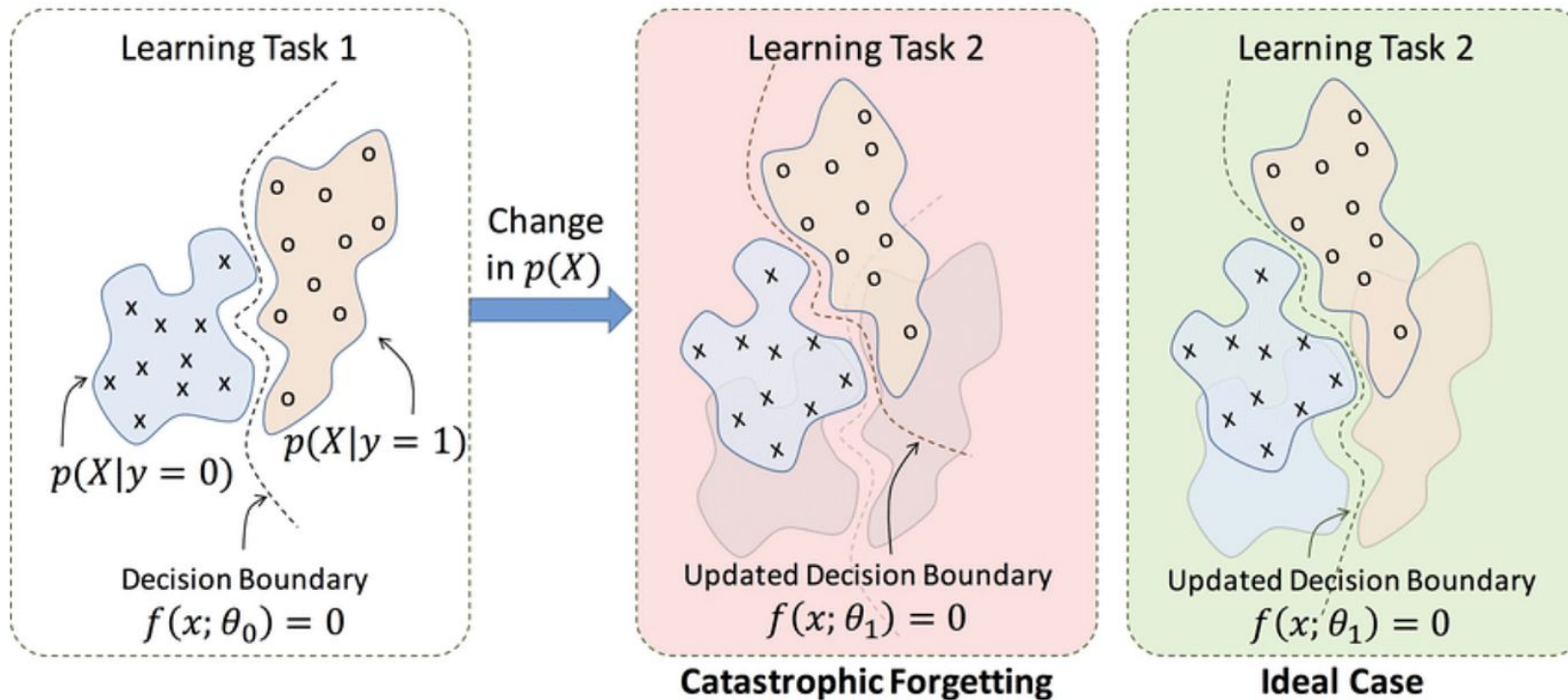
- For now, let's assume that it is possible to obtain new labeled data points from the incoming data stream
- Periodically, the classifier is ***updated*** with mini-batches of new labeled data
- We would expect that the model simply extends its knowledge considering new input data...
 - ...but life is not that simple!

A major problem: Catastrophic Forgetting

- Neural networks suffer from ***catastrophic forgetting*** of old concepts as new ones are learned.
- Learning on new data would likely override the weights that have been learned in the past
- Hence, the network may significantly degrade the recognition rate on concepts learned in the past.
- **Stability-plasticity** dilemma:
 - there is the need of a trade-off between learning new concepts and retaining previous knowledge!



Catastrophic Forgetting



[Kolouri, S., Ketz, N., Zou, X., Krichmar, J., & Pilly, P. (2019). Attention-based selective plasticity. *arXiv preprint arXiv:1903.06070*.]

The Stability-Plasticity Dilemma

- **Stability:** The capability of the model of retaining old knowledge
- **Plasticity:** The capability of the model of learning and adapting on new knowledge
- **Dilemma:** How to achieve stability without rigidity, and plasticity without chaos?
 - goal: finding a good trade-off between continuously learning but maintaining old information
 - it is a challenging open problem in AI!

Continual Learning

- Continual Learning (CL) studies the problem of learning from an infinite stream of data
 - to adapt models in new domains
 - to learn new classes
 - to personalize models
 - etc.
- **Goal:** gradually extending acquired knowledge and using it for future learning
- CL approaches aim at mitigating the catastrophic forgetting problem by balancing plasticity and stability.



Why CL is important in Ambient Intelligence?

- Adapting to changes to specific users (*concept drift*)
 - changes in health conditions
 - changes in routines/habits
 - changes in living environments/sensing setups (*feature evolution*)
- Cope with scarcity of labeled data problem
 - the HAR model is initialized with a small unrepresentative dataset
 - new labeled and unlabeled data are incrementally available over time
 - it is challenging to foresee all the possible human activities that the classifier should recognize (activity discovery)

Replay methods

- Alleviate forgetting by using “old” labeled data
- Maintaining a limited subset of stored samples (replay memory) representing old knowledge
- This subset is provided to the classifier along with new data at each incremental update
- **Pros:** low-cost, easy to implement
- **Cons:** risk of overfitting over stored samples



How to choose replay examples?

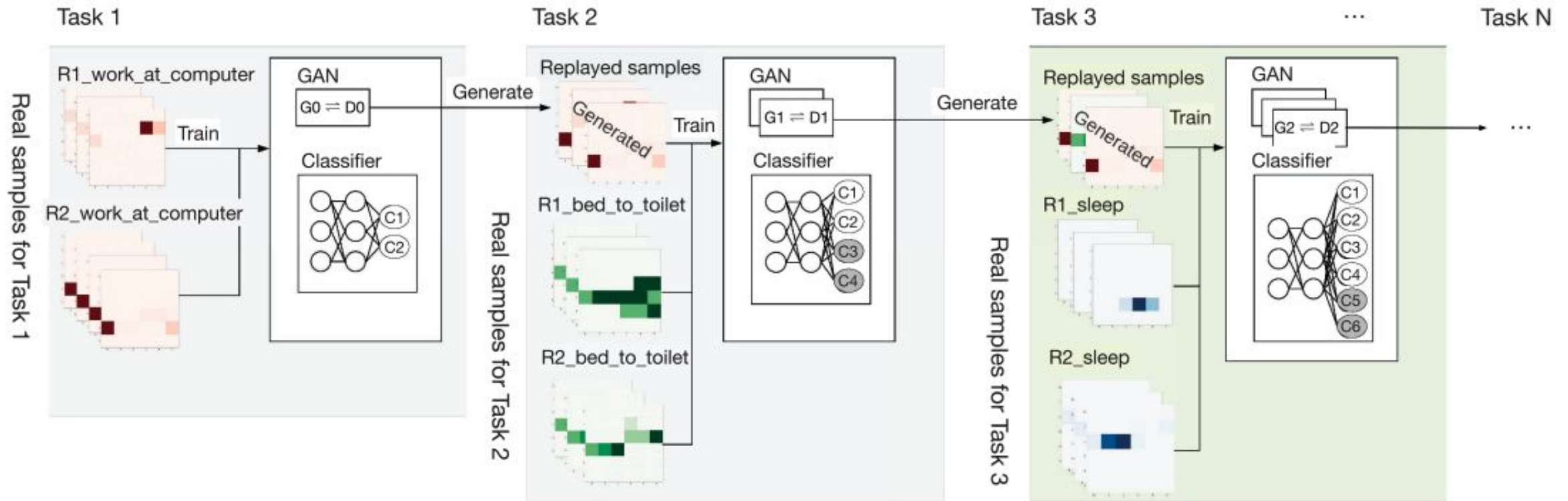
- Idea: keeping a small buffer of “old” data used to retain knowledge
- While there are many approaches, the most common are the following:
 - **Random Replay:** the buffer is populated by *randomly* sampling previous labeled data points. When learning on new data, also labeled data in the buffer are used.
 - **Exemplars Replay:** the buffer is populated with *exemplars*: representative labeled data samples (e.g., samples close to the feature mean of each class).
 - **Constrained-based Replay:** in order to retain knowledge, the last K data samples are filled in the buffer. Learning on new data is constrained by a *loss* focused on old data: it can decrease but it can not increase.
- Problems of replay-based methods:
 - requires extra storage
 - the model may overfit on the data in the buffer
 - buffer data may not be representative

Generative Replay

- Sometimes it is unfeasible to store previous samples, especially in the long time
- Idea of generative approaches:
 - The output of previous versions of the model with synthetic input is used to approximate “old” knowledge
 - e.g., Using GAN to generate high-quality data to retain knowledge
- **Cons:** complexity significantly increased!!

HAR-GAN

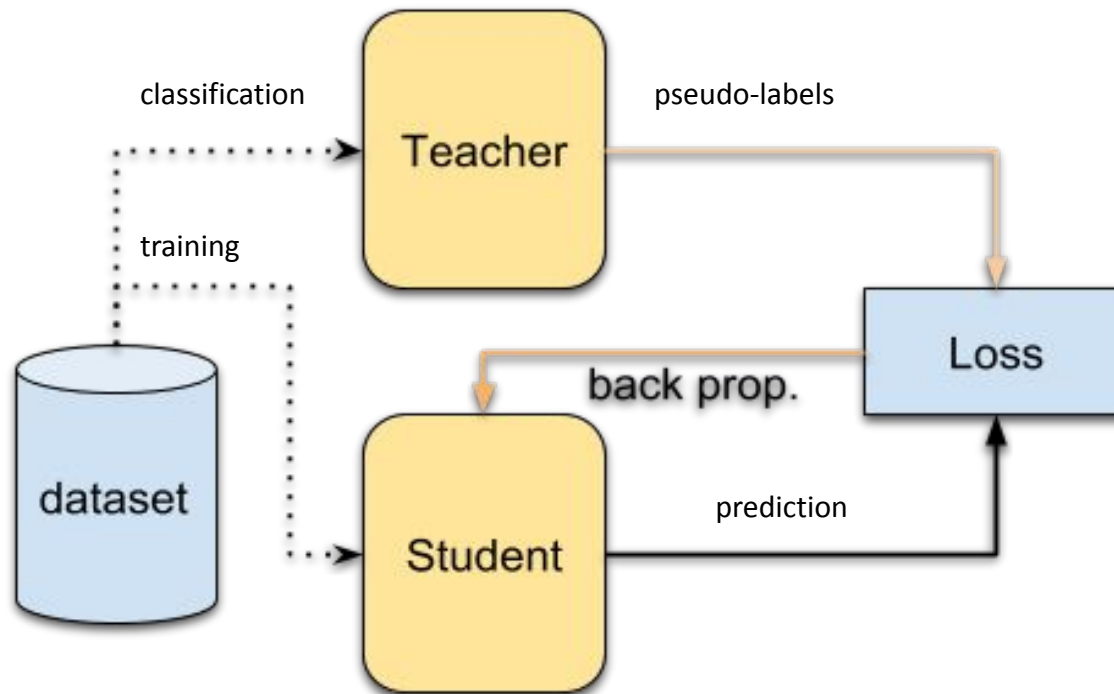
A pseudo rehearsal approach for HAR



Regularization-based methods

- Avoid storing raw inputs, prioritizing privacy
- Alleviating memory requirements
- Extra regularization terms in the loss function to consolidate previous knowledge
 - **Data-focused methods**
 - **Prior-focused methods**

A Data-Focused Regularization approach: Learning without Forgetting (LwF)



- It leverages **knowledge distillation**
- The previous model (teacher) is maintained to retain knowledge
 - it generates pseudo labels on incoming data related to old tasks.
- A specific loss function considers pseudo-labels and classification on new tasks
 - it ensures that the new model (student) learns from the teacher to output similar probability distribution on old knowledge and, at the same time, the new tasks

LwF: sketch of the approach

LEARNING WITHOUT FORGETTING:

Start with:

θ_s : shared parameters

θ_o : task specific parameters for each old task

X_n, Y_n : training data and ground truth on the new task

Initialize:

$Y_o \leftarrow \text{CNN}(X_n, \theta_s, \theta_o)$ // *compute output of old tasks for new data*

$\theta_n \leftarrow \text{RANDINIT}(|\theta_n|)$ // *randomly initialize new parameters*

Train:

Define $\hat{Y}_o \equiv \text{CNN}(X_n, \hat{\theta}_s, \hat{\theta}_o)$ // *old task output*

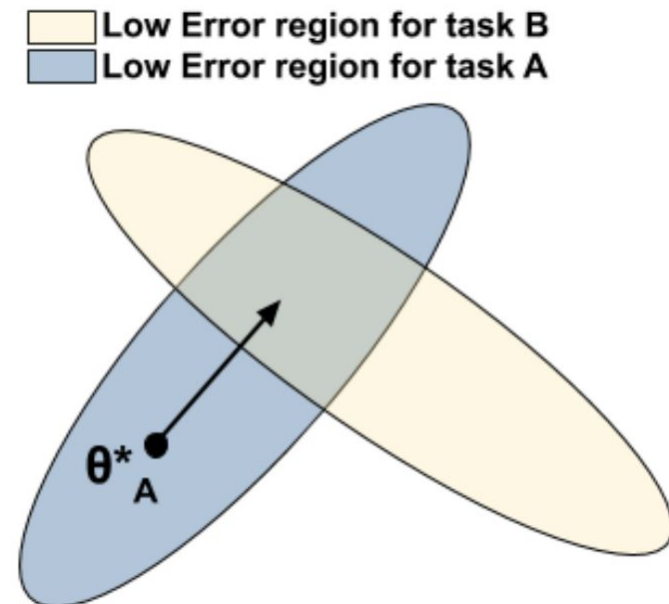
Define $\hat{Y}_n \equiv \text{CNN}(X_n, \hat{\theta}_s, \hat{\theta}_n)$ // *new task output*

$\theta_s^*, \theta_o^*, \theta_n^* \leftarrow \underset{\hat{\theta}_s, \hat{\theta}_o, \hat{\theta}_n}{\text{argmin}} \left(\lambda_o \mathcal{L}_{old}(Y_o, \hat{Y}_o) + \mathcal{L}_{new}(Y_n, \hat{Y}_n) + \mathcal{R}(\hat{\theta}_s, \hat{\theta}_o, \hat{\theta}_n) \right)$

[Li, Z., & Hoiem, D. (2017). Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12), 2935-2947.]

Regularization: Prior-focused methods

- Idea:
 - estimate an **importance value** for each weight (i.i.d. assumption)
 - while learning on new data, a **regularization term** in the loss function **penalizes** significant changes to important weights
 - hence, important weights are likely similar after training on new data, so that knowledge is maintained
- Most famous approaches:
 - **Elastic Weight Consolidation (EWC)**
 - **Variational Continual Learning (VCL)**



Desirable ingredients for Continuous Learning

- Constant memory
- Undefined set of classes
- Incremental Learning
- Few-shot learning
 - previously acquired knowledge should help for quickly learning new knowledge
- Retaining and improving acquired knowledge
- Learning also from unlabeled data
- Selective (graceful) forgetting

Semi-Supervised Learning

Semi-supervised Learning

- In real-life scenarios, it is unlikely to obtain new labels from the stream of sensor data (labeled data scarcity problem)
- A possible option to obtain new labels is to rely on semi-supervised learning methods:
 - They require a small training set to initialize the model before its deployment
 - Once deployed, the pre-trained model is used to provide labels to unlabeled data points
 - The obtained labels (also called pseudo-labels) are used to update the model
- **Goal:** reaching results close to the ones of fully supervised approaches!

Semi-supervised Learning for Personalization

- Semi-supervised incremental learning is effective in enabling personalization:
 - a model is trained collaboratively by a set of volunteers
 - the target user downloads the model locally
 - by using semi-supervised learning, the model can better adapt to the user

Semi-supervised: Static or Incremental?

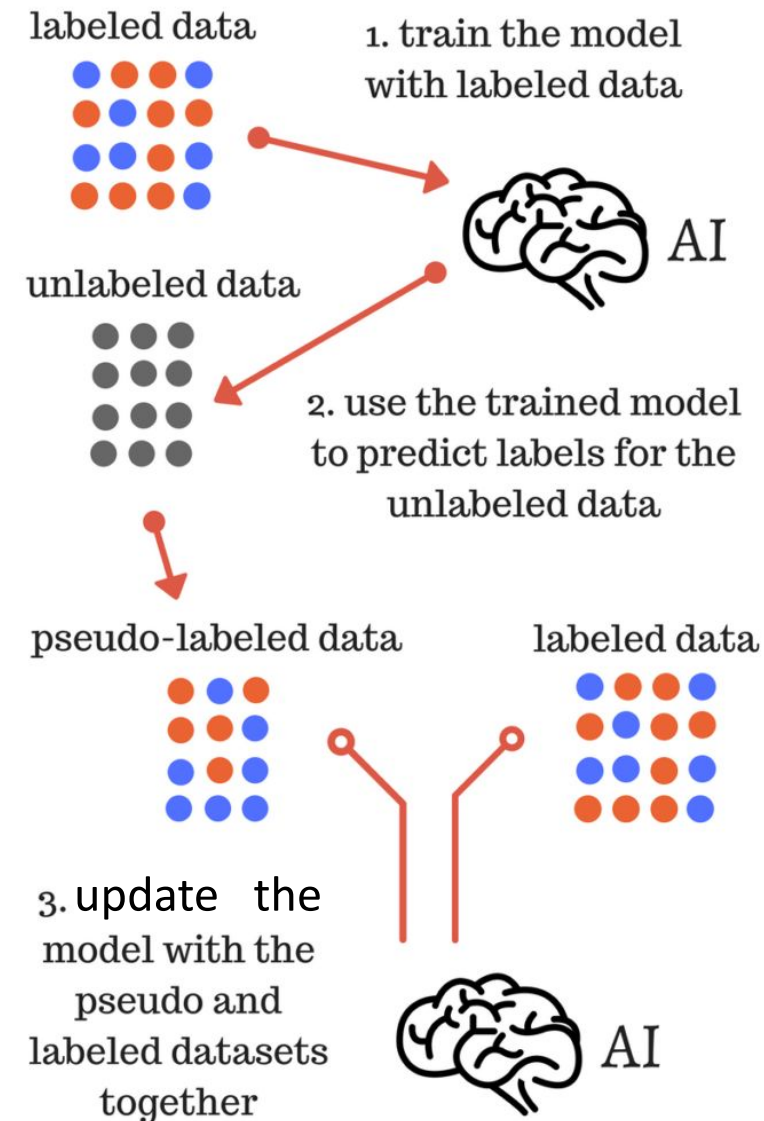
- It is important to note that semi-supervised models may also be applied to static models
 - i.e., when re-training from scratch is possible
- In this lesson, we focus on applying semi-supervised learning on the stream of sensor data in an incremental setting, since it is more realistic in Ambient Intelligence scenarios

Semi-supervised learning

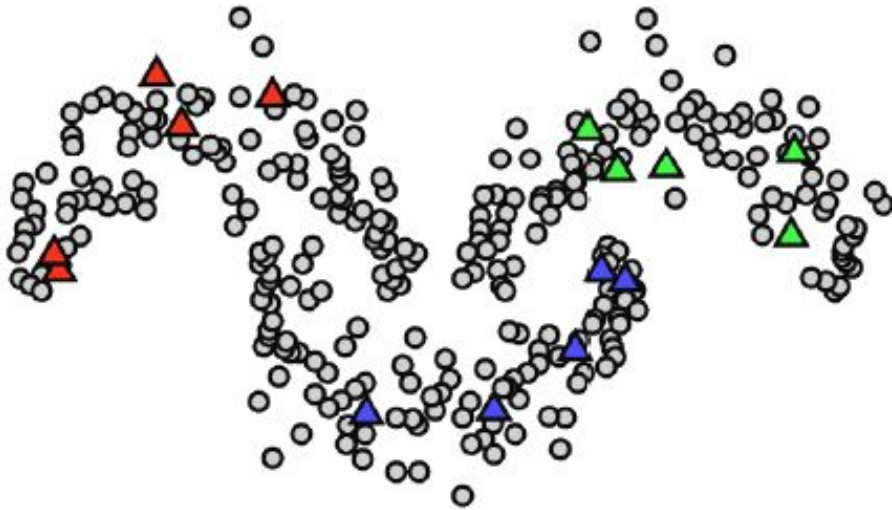
- The main semi-supervised strategies in the literature are:
 - self-training
 - co-training
 - active learning

Self-training: a basic approach

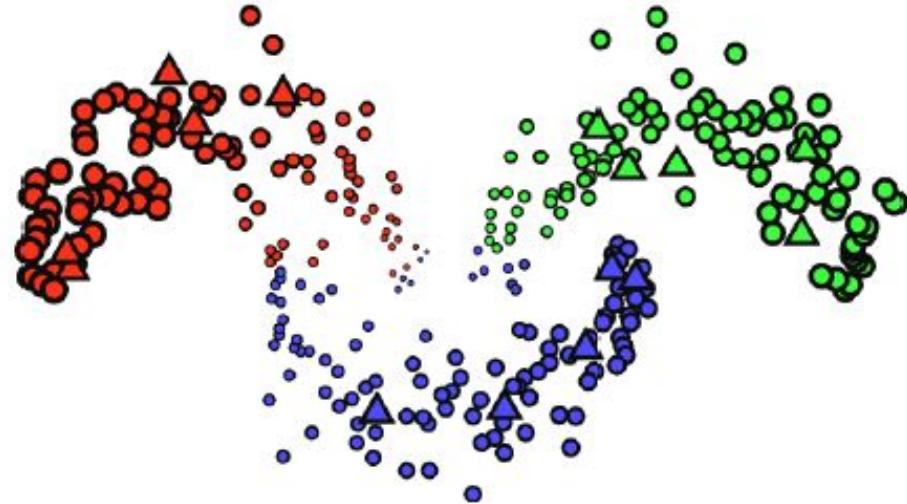
- A small labeled dataset is used to initialize the model
- The model is then used to predict labels on unlabeled data (pseudo-labels) together with the confidence level
- Only reliable predictions are actually kept as pseudo-labels
 - e.g., only when the confidence is high!
- Pseudo-labeled data are finally used to update the model



Self-Training using Label Propagation



Before label propagation: a few labeled data points and many unlabeled data points



After label propagation: labels are spreaded to nearby data points based on similarity. The size of the point indicates the propagation confidence

Label Propagation: how does it work

- Idea: Data points close in space likely correspond to the same class label
- Build a graph $G=(V,E)$
 - the nodes V are the data points (unlabeled and labeled)
 - two nodes $V1$ and $V2$ are connected by an edge that is weighted by the similarity of the corresponding data points
 - the label is spreaded from labeled to unlabeled data points based on a threshold on similarity
 - this process is repeated until convergence

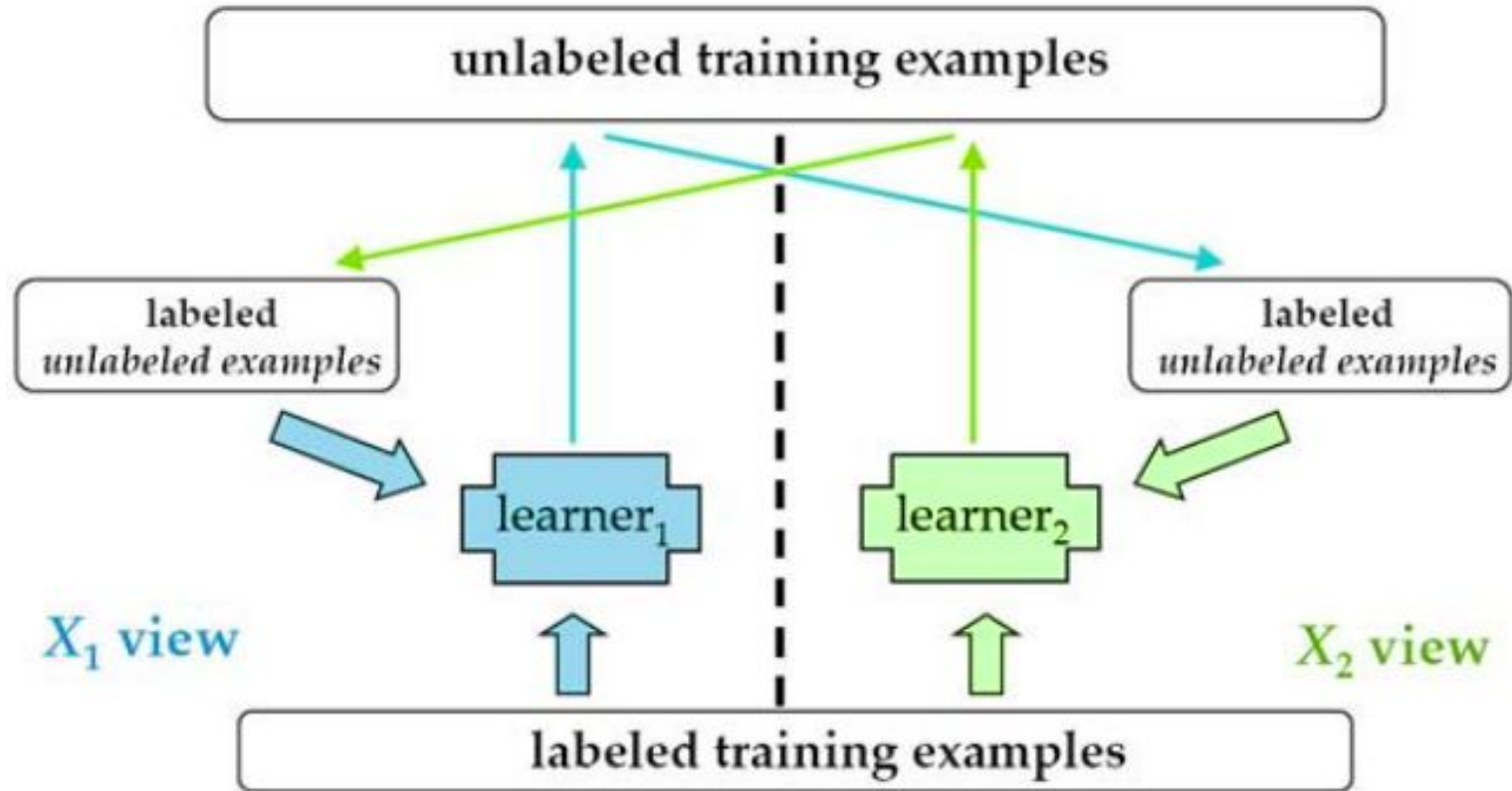
Label Propagation: cons

- Label propagation requires to store a certain amount of unlabeled data points
- It is not possible to store them all in incremental scenarios
- Hence, in an incremental setting, this algorithm could be computed periodically only considering a certain amount of historical data stored in a limited buffer

Semi-supervised Learning: Co-training

- It relies on an ensemble of classifiers, pre-trained on available labeled data
- Each classifier is trained on a different **view** of the dataset
 - e.g., different feature vectors, different portions of the dataset, ...
- On each unlabeled data point, each model performs its prediction
- Each model uses the most confident predictions from the other models as additional (pseudo-)labeled data samples to improve its performance on its own view of the data

Co-training



Co-training cons

- It can be less effective when the views of the data are not complementary
 - the design of data views is not trivial!
- Similarly to self-training, errors in one model may be propagated to the other models, leading to a decline in performance.

Active learning

- A semi-supervised approach widely used in Ambient Intelligence
- It requires the **interaction** with the user
- When the model is “uncertain” about a specific output it triggers a query to the user
 - e.g., by analyzing the probability distribution during classification
- The user has to interact with the system to provide the ground truth
 - the new labeled data point is then used to update the model!
- Such methods should minimize the number of triggered queries as much as possible

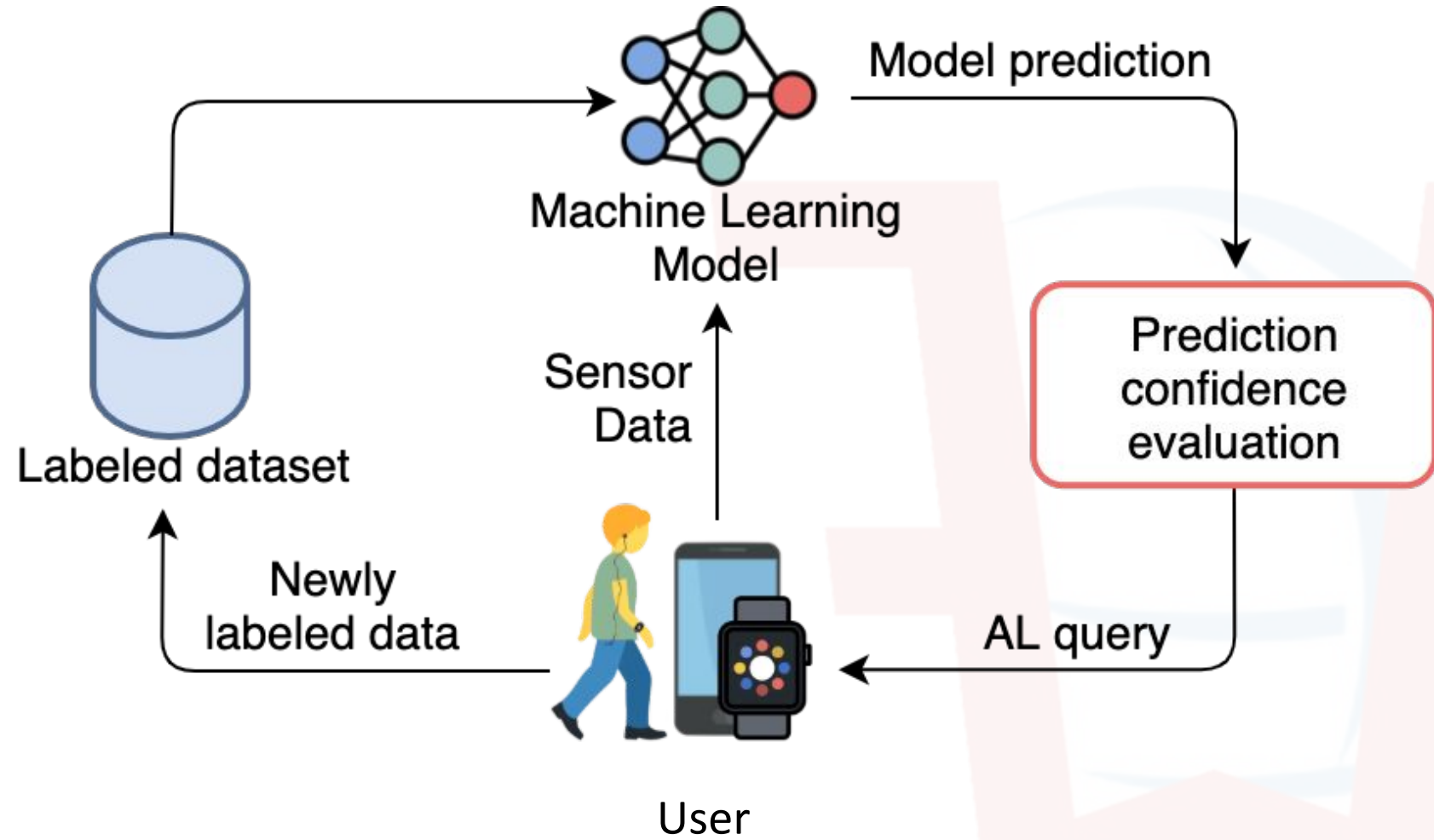


Active Learning: the idea

The stream of unlabeled data is provided to a pre-trained machine learning model to obtain predictions

When needed, an active learning query is triggered

The model is updated with mini-batches of data labeled from the user



Uncertainty Measures

- The goal of Active Learning is to trigger a query only for the most ***informative*** data points, the ones where the classifier is the most uncertain
- In the following, we will see the most common ones

Uncertainty Measure: Least Confidence

- The simplest uncertainty measure to implement for Active Learning
 - it relies on the confidence on the most likely class
 - idea: when the confidence on the most likely class is below a certain threshold, it means that the classifier is uncertain

$$\phi_{LC}(x) = 1 - P_{\theta}(y^*|x)$$

Uncertainty Measure: Smallest Margin

- Instead of considering only the most likely activity, compute the difference between the two most confident predictions.
- Rationale: how much more confident was the model on the most likely class than for the next-most-confident class?

$$\phi_M(x) = P_\theta(y_1^*|x) - P_\theta(y_2^*|x)$$

Uncertainty Measure: Entropy

- Goal: consider the whole probability distribution
- How? Computing the *entropy* measure:
 - a low entropy value indicates that there is one class with high probability and the others with low probability
 - a high entropy means that more activities have similar probability values
- The user is queried when the entropy is higher than a threshold

$$-\sum_i P(y_i|x; \theta) \log P(y_i|x; \theta)$$

Entropy and Active Learning: an example

Suppose that there are three activities: A1, A2, A3

Example 1:

- the probability distribution in output is $[0.8, 0.15, 0.05]$
- intuitively, it is clear that A1 is the most likely and a query is not necessary
- entropy value: ~ 0.61

Example 2:

- the probability distribution in output is $[0.5, 0.4, 0.1]$
 - here, the classifier is not so sure about A1, a query may be useful to refine the model
- entropy value: ~ 0.94

Active Learning: Adaptive threshold

- Once the uncertainty measure is chosen, determining an active learning threshold may be challenging in real-world scenarios
- There are methods to dynamically adjust it based on the input data
- Idea:
 - the threshold is initialized with a starting value
 - when the system is ***uncertain***, the threshold should be adjusted to trigger more queries to refine the model (the uncertainty region is decreased)
 - if the system is ***certain***, it means that the threshold can be adjusted to increase the uncertainty region and trigger less queries

Adaptive Active Learning: The VAR-UNCERTAINTY strategy

Algorithm 4 Strategy VAR-UNCERTAINTY(X_t, L, s)

Input: incoming instance X_t , trained classifier L ,
threshold adjustment step $s \in (0, 1]$

Output: *labeling* $\in \{\text{true}, \text{false}\}$

Initialize: initialize labeling threshold $\theta \leftarrow 1$ and store
the latest value during operation

```
1  $\hat{y}_t \leftarrow \arg \max_y P_L(y|X_t)$ , where  $y \in \{1, \dots, c\}$ ;  
2 if  $P_L(\hat{y}_t|X_t) < \theta$  then  
   | //uncertainty below the threshold  
3   | decrease the uncertainty region  $\theta \leftarrow \theta(1 - s)$ ;  
4   | return labeling  $\leftarrow \text{true}$   
5  
6 else  
   | //certainty is good  
7   | make the uncertainty region wider  $\theta \leftarrow \theta(1 + s)$ ;  
8   | return labeling  $\leftarrow \text{false}$ 
```

Note that, this strategy uses the *least confident* uncertainty measure, but it can be adapted with any measure.

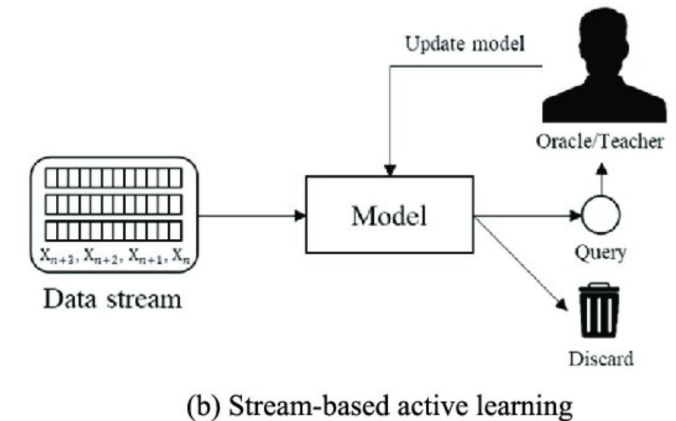
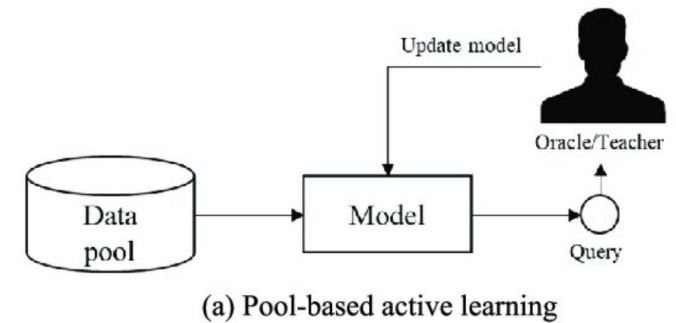
While *least confident* may work well with a few classes, entropy is sometimes more robust when there are several classes (e.g., in HAR).

VAR-UNCERTAINTY can be easily adapted with other uncertainty measures by changing the condition at line 2.

[Žliobaitė, I., Bifet, A., Pfahringer, B., & Holmes, G. (2013). Active learning with drifting streaming data. *IEEE transactions on neural networks and learning systems*, 25(1), 27-39]

Pool-Based vs Stream-Based Active Learning

- In the general machine learning domain (e.g., Computer Vision), active learning is used in a pool-based fashion
 - i.e., there is a large pool of unlabeled data and, in an offline phase, the system chooses a number of data points maximizing uncertainty measures
- In Ambient Intelligence, it is likely impossible that a human can label a sensor data sample long after it is collected
- Hence, we consider a *stream-based* active learning, where uncertainty is evaluated on each new sensor data sample
 - the user should be triggered with an active learning query as soon as the model is uncertain!

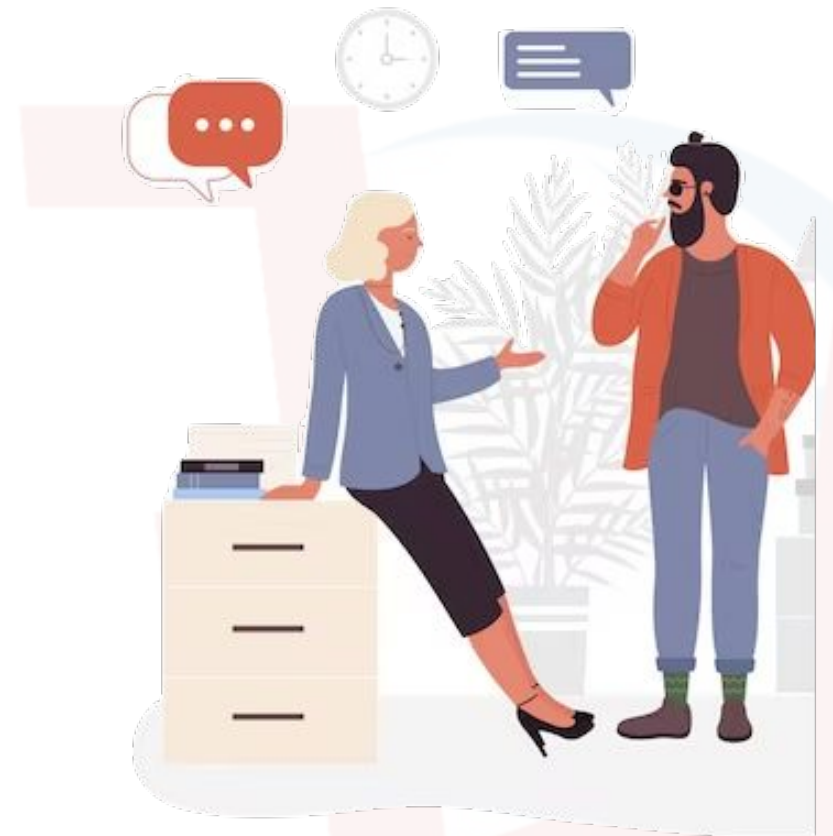


Acceptability of Active Learning

The majority of Active Learning methods in the literature assume that **users are constantly available** to provide a feedback

In **real-world** scenarios this is **not realistic**:

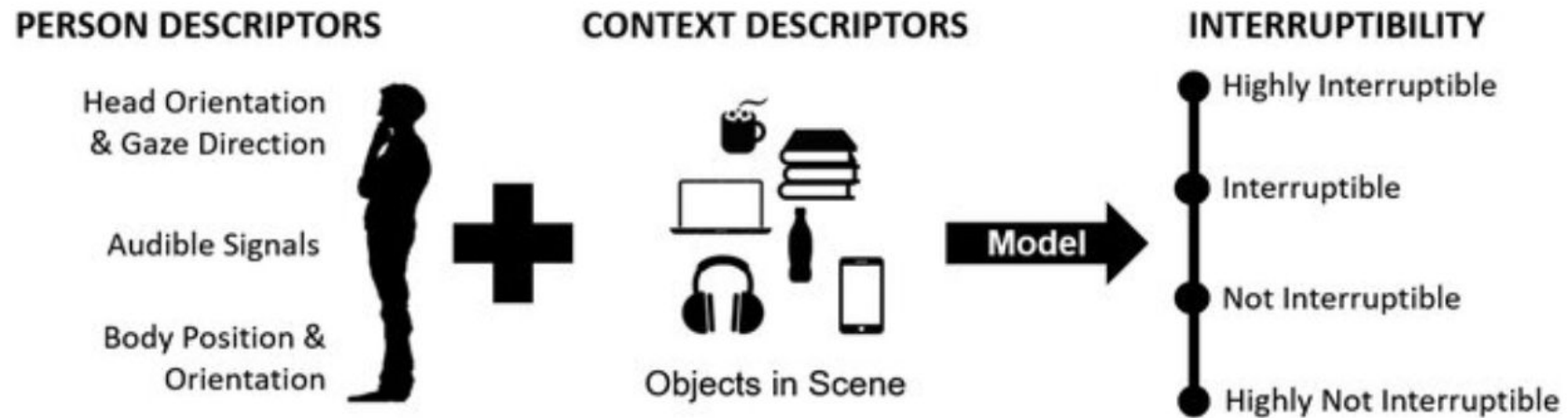
- Notifications in **inappropriate moments** may lead to:
 - **Stress and frustration**
 - Increasing **the feedback error rate**
 - Disaffection with the system



Towards Active Learning in real-world scenarios

- Queries should be triggered considering the **context** of the user
 - e.g., current activity, location, social context, how many queries received in recent past
- It is necessary automatically collect and process context data suggesting when the users are interruptible and when they are not
- Also, users should be **incentivized** to provide labels
 - e.g., gamification, economic rewards, promise of a better model
- These are still open research problems and poorly investigated!!!

Interruptibility Models



Summarizing limits of semi-supervised approaches

- ***Self-training:***
 - errors in label spreading may be amplified with time
 - hard to spread labels to “different” data w.r.t. the ones used for training
- ***Co-training:***
 - even though it is more robust than self-training, it shares similar problems
 - moreover, it is challenging to design different effective views
- ***Active learning:***
 - while it is the most effective one, it requires the interaction with the user which may not be always possible

Activity Discovery

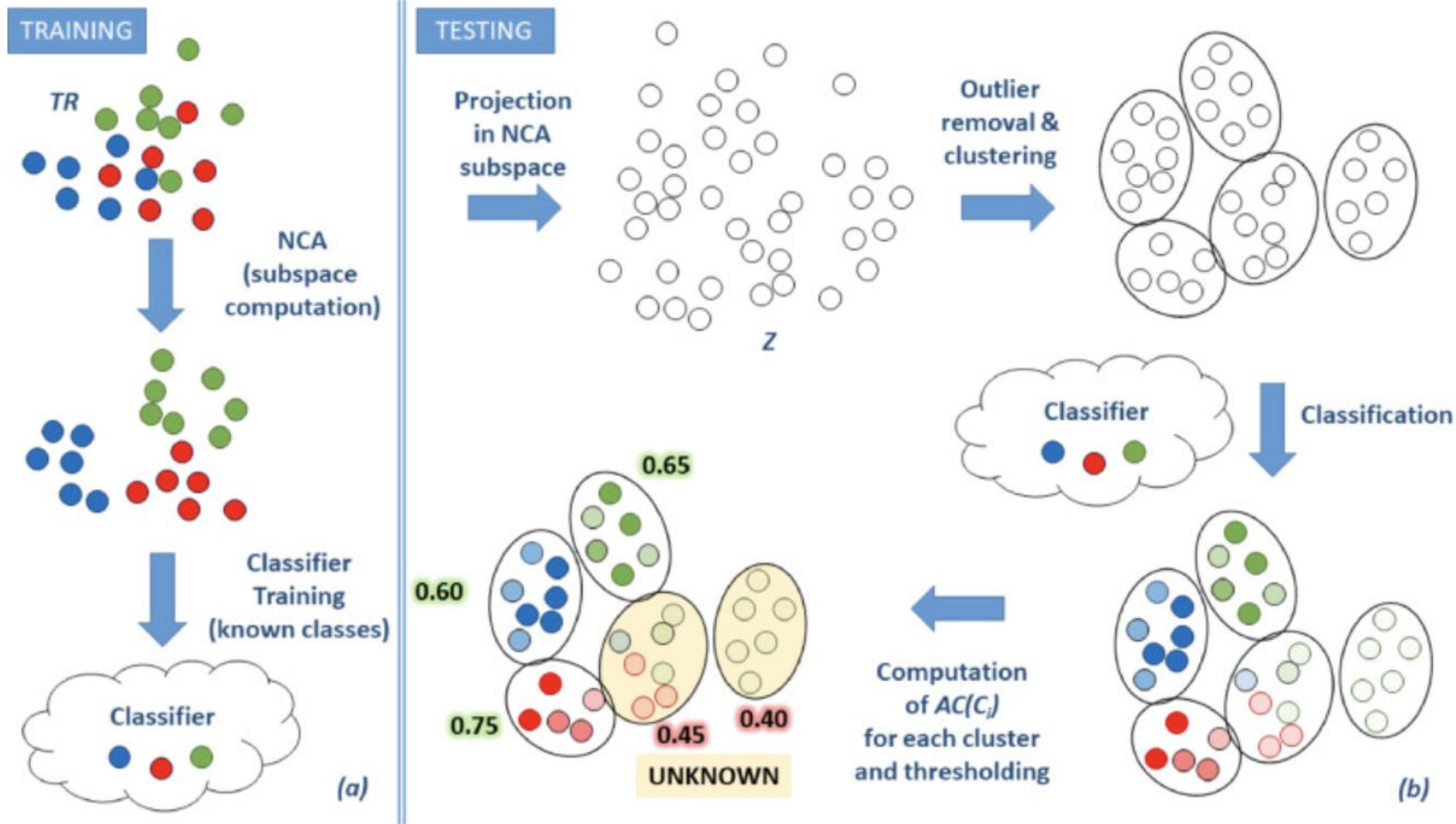
Activity Discovery

- Until now, we assumed a fixed set of activities
- However, it is not possible to foresee all of them in advance.
- Different users may perform different types of activities
- Semi-supervised learning may enable activity discovery!
- Idea: pre-training the model using a dataset with fixed activities, and applying **activity discovery** methods to automatically find new patterns from unsupervised data points
- New patterns may indicate
 - new activities
 - different ways of performing previously considered activities

Activity Discovery: how?

- Idea:
 - perform clustering on unlabeled data points from the stream
 - discover reliable patterns
 - use the pre-trained model to annotate clusters (leveraging self-training methods)
 - when the model is uncertain in classifying data on a specific cluster, it may be a new activity!
- The annotation of new clusters can be based on Active Learning

Semi-Supervised Activity Discovery

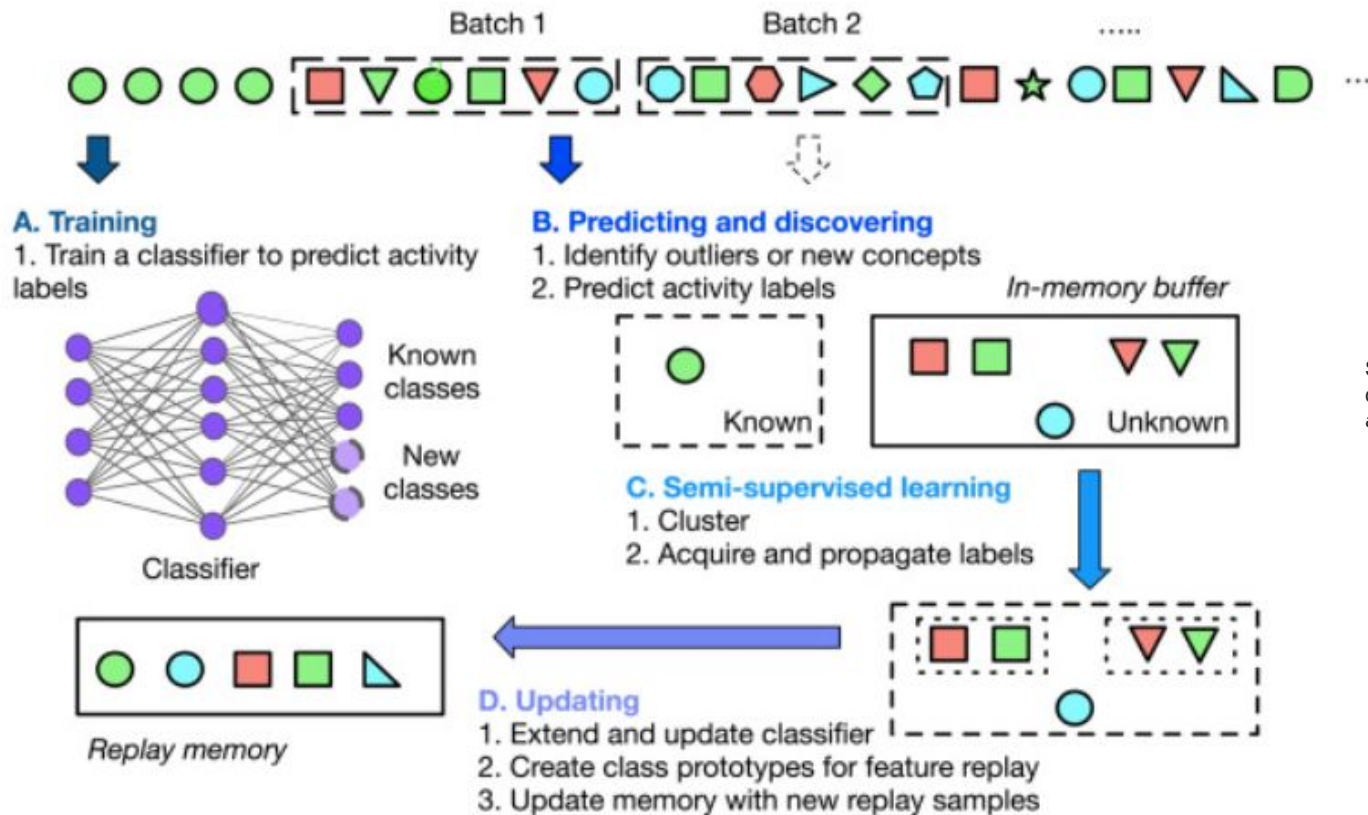


[Brighi, M., Franco, A., & Maio, D. (2021). ActivityExplorer: A semi-supervised approach to discover unknown activity classes in HAR systems. *Pattern Recognition Letters*, 151, 340-347.]

Incremental Activity Discovery

- Performing activity discovery in incremental settings is significantly more challenging
- Besides requiring a pre-trained model, it is also necessary to memorize information about the original data distribution to avoid catastrophic forgetting
- On the stream of sensor events, it is necessary to find sensor patterns that deviates from the original distribution (new concepts)

Continual Learning with new activities: A Replay Approach



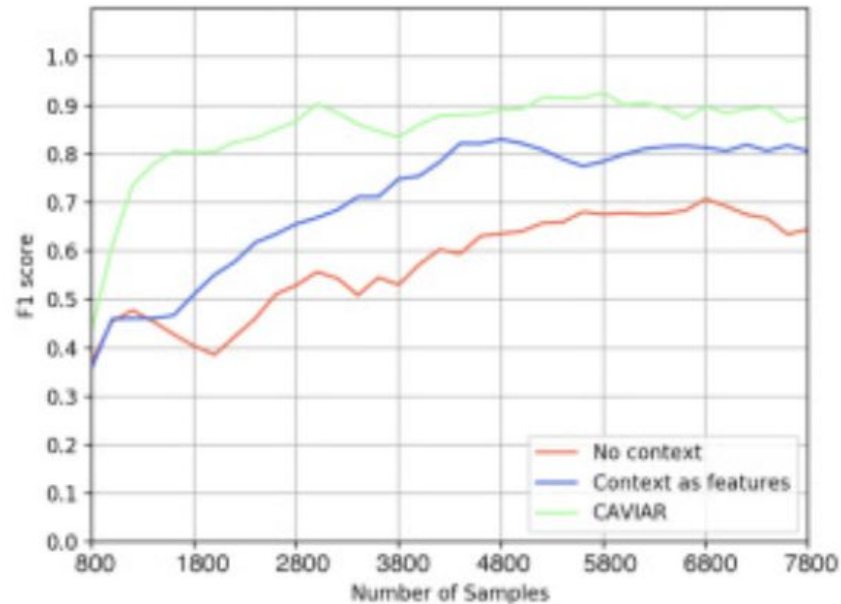
Schiemer, M., Fang, L., Dobson, S., & Ye, J. (2023). Online continual learning for human activity recognition. *Pervasive and Mobile Computing*, 93, 101817.

Evaluating Incremental Classifiers

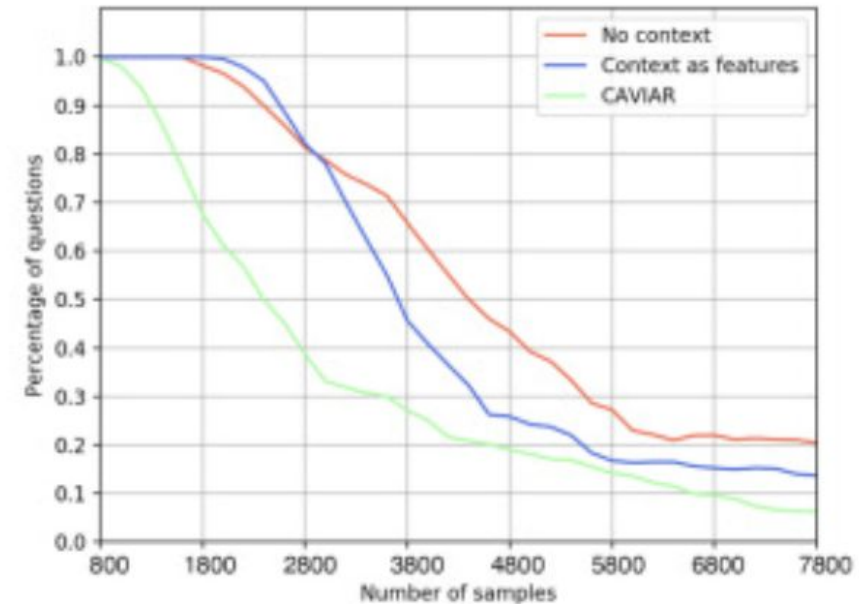
How to evaluate incremental classifiers?

- Since incremental classifiers continuously learn from data, it is crucial to evaluate the **evolution** of the recognition rate
- For each data sample in the stream:
 - use the current model to classify it
 - apply a semi-supervised learning method (e.g., evaluate if an active learning query is needed)
 - store the obtained model prediction, the ground truth, and the pseudo-labels
 - memorize if an active learning query was triggered
 - update the model when needed (e.g., if a sufficient number of pseudo-labels was collected)
 - in this last step, Continual Learning strategies may be considered
- Visualize the evolution performing a sliding window of the predictions:
 - for each window it is possible to compute the metrics of interest (e.g., F1 score of the model, number of queries)
 - the size of the window determines the periodicity of model evaluation. It should be high to make sure that the captured samples are representative

Evaluating incremental classifiers: An example of visualization



(a) F1 score



(b) percentage of questions

References

- [1] Abdallah, Z. S., Gaber, M. M., Srinivasan, B., & Krishnaswamy, S. (2018). ***Activity recognition with evolving data streams: A review***. ACM Computing Surveys (CSUR), 51(4), 1-36.
- [2] De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., ... & Tuytelaars, T. (2021). ***A continual learning survey: Defying forgetting in classification tasks***. IEEE transactions on pattern analysis and machine intelligence, 44(7), 3366-3385.
- [3] Gama, J., Sebastiao, R., & Rodrigues, P. P. (2013). ***On evaluating stream learning algorithms***. Machine learning, 90, 317-346.